PG-MDP: Profile-Guided Memory Dependence Prediction for Area-Constrained Cores

MICRO 2026 Submission **#1358** – Confidential Draft – Do NOT Distribute!!

Abstract

Memory Dependence Prediction (MDP) is a speculative technique to determine which stores, if any, a given load will depend on. Area-constrained cores are increasingly relevant in various applications such as energy-efficient or edge systems, and can often only spare limited space for MDP tables. This leads to a high number of false dependencies as memory independent loads collide with unrelated predictor entries (i.e., the aliasing problem), causing unnecessary stalls in the processor pipeline.

The conventional way to address this problem is with greater predictor size or complexity, but these are costs which cannot be afforded on area-constrained cores. This paper proposes that targeting the **predictor working set** is as effective as growing the predictor, and can deliver performance gains which match that of very large predictors while still using very small predictors. This paper introduces profile-guided memory dependence prediction (PG-MDP), a software co-design to label consistently memory independent loads via their opcode and remove them from the MDP working set. These loads are referred to as *predict-no-dependency* (PND) loads, and bypass querying the MDP when dispatched, always issuing as soon as possible. Across SPEC2017 CPU intspeed, PG-MDP reduces dynamic MDP queries by 77%, false dependencies by 80%, and improves IPC for a small simulated core by 1.47% (to within 0.5% of a very large predictor), all at **zero hardware cost**.

1 Introduction

-todo: comments (overleaf, paul (and abstract), jacky) mention ISA hint and detail PGO regen graphs fill in sweep and power sections redo section overviews and conclusion figures

Memory dependence prediction (MDP) is a speculative technique used in out-of-order processors to increase available instruction-level parallelism (ILP) by predicting the stores on which a given load instruction will depend, as well as identifying loads that are memory independent. Smaller cores utilising smaller predictors can often struggle with a large number of false dependencies, which unnecessarily stalls loads on non-dependent stores. This is due to hash collisions in small predictor tables, or simple predictor algorithms that cannot capture varying dependency behaviour. The Store Sets predictor [2] exhibits both of these issues, and variations of the algorithm are found in real processors today [**?**]. Whereas modern predictors like PHAST [5] have pushed the state of the art to address these issues via techniques like associative tagging and context based predictions, they place higher area, energy and complexity demands on the core design, which is often not viable in smaller cores. While simply growing a Store Sets-like predictor would offset hash collisions, the potential benefits of larger memory dependence predictors is bounded by the total available ILP, often making this an inefficient investment in cores with short instruction windows. As such, memory dependence predictors in area-constrained cores typically remain small and inaccurate.

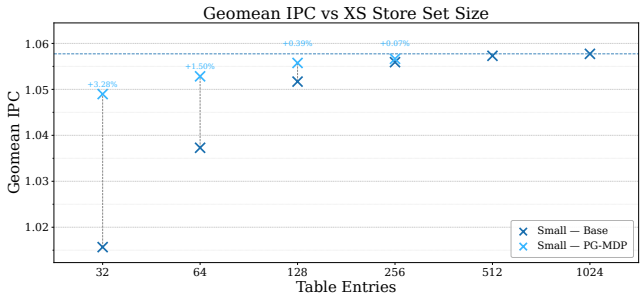

Figure 1: Base and improved IPC for increasing XS Store Set [?] sizes on a small core (ROB=128). Past 128 entries, performance gains quickly diminish. With PG-MDP, 64 entries deliver within 0.5% of available IPC from 1024 entries, and 128 entries deliver almost all available IPC. PG-MDP plots past 256 entries are omitted as performance is near-identical.

Nevertheless, the importance of such area-constrained cores has grown over time. Modern systems increasingly deploy heterogeneous processors containing both performance and efficiency-oriented core designs, combining larger high-performance cores with small energy-efficient (but still out-of-order) cores, as seen in ARM big.LITTLE systems and both Apple's & Intel's performance and efficiency cores. These efficiency cores share die area with high-performance cores under strict budgets for both area and power. Crucially, this pattern is no longer confined to mobile devices, and this heterogeneous design can now be found across mobile, desktop and server designs. Even as processors in high-end mobile devices scale to rival those found in laptops, they are often paired with efficiency cores to maximize battery life. Area-constrained cores are further found as standalone cores in edge systems, or efficiency tiles in a disaggregated chiplet architecture. In all these cases, out-of-order execution components are scaled to fit the given area envelope rather than to maximize performance. As these area-constrained cores are increasingly tasked with demanding general-purpose computation while striving to consume as little energy as possible, increasing ILP without incurring additional area, power or complexity costs becomes more pressing.

In this paper we argue that the problem of false memory dependencies is not necessarily a question of predictor size or complexity, and that, for area-constrained cores, it is the **predictor working set** that is most important, and that for an appropriate working set even a very small predictor is able to deliver most of the performance of a very large predictor. We define the predictor working set as the set of all instructions that actively index into the predictor

MICRO 2026, Athens, Greece
2026. ACM ISBN 978-X-XXXX-XXXX-X/XX/XX
https://doi.org/XXXXXXX.XXXXXXX

during a given execution window. For Store Sets, this includes all issued loads and stores, whether or not they are actually memory dependent. Prior work has demonstrated that a considerable fraction of load instructions in general-purpose workloads rarely or never exhibit in-flight dependencies [**? ? ?**]. Hence, these loads represent ideal candidates to be removed from the MPD working set.

We exploit this insight with **profile-guided memory dependence prediction (PG-MDP)**, a software co-design at **zero hardware cost** which identifies frequently memory-independent loads and labels them via their opcode as *predict no dependency* (PND), causing them to bypass MDP queries entirely, and always schedule as early as possible. In the event that a labelled load incorrectly passes a store to the same address, a rollback is triggered as usual, but no new MDP entry is created. PG-MDP reduces average runtime MDP queries by 77% and false dependencies by 80% across SPEC2017 CPU intspeed. Crucially, by reducing the working set to only genuinely memory dependent loads, very small Store Set-like predictors are able to match the performance of much larger ones: on a small simulated core, PG-MDP improves geomean IPC of a 64 entry XiangShan Store Sets predictor [**?**] by 1.47%, falling within 0.5% IPC of a 1024 entry predictor, as shown in Figure 1.

1.1 Contributions

This paper makes the following contributions:

- **Reframes MDP Aliasing:** We show that for Store Sets-Like predictors, the dominant source of false dependencies does not stem from insufficient predictor capacity per se, but rather from an unnecessarily large working set that captures both memory dependent and independent loads. We demonstrate that framing MDP aliasing as a capacity problem is misleading, and that shrinking the working set is as effective as growing the predictor.
- **Introduces PG-MDP:** By leveraging profile-guided memory dependence prediction, we reduce dynamic MDP queries by 77% on average, across Spec2017 CPU intspeed, and false dependencies by 80% with zero hardware cost.
- **Closes Store Sets Table Size Gap For Free:** By shrinking the MDP working set to only memory dependent loads, PG-MDP allows a 64-entry XiangShan Store Set predictor to match the performance of 1024 entries within half a percent, achieving a geomean 1.47% IPC gain on a small simulated core.

2 Background

This section outlines the fundamental concepts in memory-level parallelism for out-of-order execution. It then introduces Store Sets as a foundational memory dependence predictor algorithm, a modern variation found in the XiangShan processor, and lastly the current state of the art in the PHAST predictor.

2.1 Loads in Out-of-Order Execution

A key structure in out-of-order execution is the load-store queue (LSQ), used to hold in-flight memory operations and perform memory disamgibuation. When load instructions dispatch, they are inserted into the load queue (LQ), and query the memory dependence predictor (MDP) for a predicted store(s) to wait on before execution. Once their source operands are ready and all predicted dependent stores have executed, loads search backwards through the store queue (SQ) for store-forwarding opportunities. If no matching addresses are found, the loads access cache to get their data.

When a load searches the SQ, not all earlier stores may have resolved their addresses. As the majority of the time the address will not match with the load, it is often profitable to speculatively ignore these stores and issue the load as soon as possible. When the unresolved store eventually receives it's address, it searches forward through the LQ to find loads with matching addresses which have already executed. If a match is found, the load has read stale data and a memory order violation has occured. The processor must flush all in-flight instructions from the load onwards and re-fetch.

The goal of the MDP is to minimise violation events while maximising the available ILP, allowing memory independent loads to issue as soon as possible and dependent loads to wait only for the necessary stores. When a load is incorrectly stalled on a store that does not resolve to the same address this is called a false dependency. Simple MDP algorithms typically prioritise reducing the number of violations over the number of false dependencies.

2.2 Store Sets

A seminal paper in memory dependence prediction is 'Memory Dependence Prediction using Store Sets' [2]. This is the MDP algorithm implemented in the open source Gem5 simulator [3]. Store Sets works by using Store Set IDs (SSID) to track dependent load-store pairs, assigning each instruction the same ID value in the PC-indexed Store Set ID Table (SSIT). The SSIDs are then used to index the Last-Fetched Store Table (LFST), which holds the sequence number of the most recently issued store with an SSID mapping to that LFST entry. Newly issued loads then access the LFST via their SSID to find the store they're predicted to be dependent on. If a load PC does not map to a valid SSID entry, it is predicted to be memory independent. To forget old dependencies and reduce table pressure, the SSIT and LFST are cyclically cleared after a certain number of issued memory operations.

Store Sets is extremely area efficient, delivering a large portion of available performance with a tiny hardware budget. It's small size and simplicity make it well suited to area-constrained processors. However, especially at smaller sizes, Store Sets struggles with false dependencies due hash collisions. When a memory independent load hashes to an existing SSID entry, it is incorrectly made dependent on the corresponding store for that entry. The clear period can be made shorter to offset this, but this comes at the trade-off of a higher number of memory order violations as the predictor must re-learn true dependencies more often. Furthermore, many loads exhibit non-consistent memory dependencies. A load in a loop may be dependent on a store every even iteration, but not on odd iterations. Store Sets has no way to disambiguate these cases, and conservatively predicts the load as dependent on every iteration.

2.3 XiangShan Store Sets

XiangShan is an open-source high-performance RISCV processor RTL implementation [**?** **?**]. XiangShan represents the cutting edge in open-source superscalar processor design, and comes with a Gem5 fork modified to closer model the RTL implementation [**?**]. This Gem5 fork implements a variation of the Store Sets predictor that offers a closer representation to implementations found in real processors. From here on this is referred to as XS Store Sets.

The main optimisation over original Store Sets is using multiple 'slots' for each LFST entry, thereby recording multiple dependent stores at once. This allows a load to track multiple dependencies with only one SSID. This is useful in situations where a load is dependent on different stores in different program contexts, or may have partial address overlap with multiple stores. There are further small optimisations to allocation policy which we omit here.

2.4 PHAST

PHAST is a newer predictor which achieves much greater accuracy than Store Sets and other previously proposed MDP algorithms [5]. PHAST uses a TAGE-like structure to record a geometric series of branch history lengths between dependent load-store pairs. On dispatch, load instructions search all history length tables in parallel and selects the dependencies on the longest path (or predicts no dependency if no tables hit). This scheme addresses the previous problems in Store Sets by using multi-way tagged associative entries to reduce hash collisions, and predicting dependencies based on branch context to capture more elaborate dependency patterns. At a storage budget of 14.5KB, PHAST is shown to achieve within 1.5% accuracy of an ideal predictor.

Although PHAST delivers much greater accuracy, it also comes with greater hardware overhead. Though PHAST greatly outperforms Store Sets for iso-area comparisons at larger budgets, it struggles with the smaller budgets most efficiency-focused cores allow for memory dependence prediction. Furthermore, PHAST incurs pipeline complexity by requiring global branch history be made available in the load-store unit, has longer prediction latency, and consumes more power for the same area. This makes PHAST better suited only for larger performance-focused cores that would benefit most from the greater ILP.

3 Method

This section presents the profile-guided memory dependence prediction (PG-MDP) technique. We put forward the concepts of store distances, distance thresholds, and how these are combined to find reliably memory independent loads. We then motivate the use of profiles to determine load labels, outlining the benefits provided over static analysis.

The PG-MDP workflow follows a standard profile-guided optimization pattern. Programs are compiled AJ: Are they instrumented? once then profiled using train inputs. During profiling, PG-MDP records the dependent store for each load AJ: How is this achieved, through a cross-product of all loads with all stores check?. Loads which have sufficiently long 'store distances' (detailed below) at least 95% of the time are labelled *predict no dependency* (PND) loads, and during recompilation their opcodes are changed appropriately. PND loads then skip querying the memory dependence predictor (MDP) at run time and are always issued as soon as possible. If they really do observe a dependency and trigger a memory order violation, this still causes a rollback as normal but no new entry is added in the MDP.

3.1 Store Distances & Labelling Thresholds

As overviewed in Section **??**, when loads are issued in out-of-order execution they search the store queue (SQ) for older stores with overlapping addresses. Informally, the term *store distance* of a load refers to the number of SQ entries that must be checked to locate the load's corresponding store. For instance, a load depending on the most recent prior store has a store distance of 1. Formally, let x be the address of a load, and let the SQ contain store addresses $y_1, y_2, \ldots, y_n$ ordered from youngest (y_1) to oldest (y_n). The *store distance* is defined as the minimum index i such that $x = y_i$. If no such index exists, the store distance is ∞.

The profile-derived store distance for each load is found by recording every per-execution store distance on train inputs. Store distances that occur in 95% or more executions across all train inputs are selected as that load's profiled store distance. When no single store distance occurs in at least 95% of executions, the shortest observed distance is chosen.

Loads are then filtered by whether their profiled store distance is within a certain threshold. The premise PG-MDP builts on is that loads with either ∞ or sufficiently long dependent store distances are good candidates to label as PND. This heuristic captures four different types of behaviours:

- **(1) Memory independent loads:** Loads which read constant data (i.e. have no store distance).
- **(2) Rarely dependent loads:** Loads which observe short dependent store distances in <5% of executions.
- **(3) Structurally independent loads:** Loads with no dependent stores in-flight at the time the load is issued.
- **(4) Fast-to-Resolve Dependencies:** Loads which have in-flight yet resolved dependent stores, allowing forwarding.

Loads with behaviour type (1) are always labelled regardless of the threshold value. Whether a load exhibits behaviours (2) to (4) depends on the target processor. For instance, a processor with a longer SQ will hold more in-flight stores at once, so only loads with longer store distances will be labelled. The proportion of loads that exhibit type (4) behaviour is even further target specific, but is still correlated with a longer store distance as this puts more time between dispatching the store and issuing the load, making it more likely the store's address will be resolved when needed.

The optimal store distance threshold for a specific target processor is found via binary search, choosing the threshold with the best average performance over train inputs. By using a representative selection of workloads to perform the search with, a single optimal store distance threshold can be found for the target processor rather than individual thresholds for each workload, meaning the search only has to be performed once.

3.2 Why Profiles?

While profile-guided optimisation (PGO) has long offered performance benefits, the technique has historically struggled to achieve adoption due to complicating the compilation workflow [**?**]. PGO

also introduces concerns about the accuracy of generated profiles, and the potential to harm performance should real input differ too much from the train input.

Addressing adoption, prior work shows that use of PGO has risen significantly in recent years [**?**]. This can be seen in enterprise projects such as Firefox and Chrome web browsers, the Python interpreter and the GCC compiler. Performance-critical server workloads have also seen renewed attention to the benefits that PGO can provide [**?**]. This enforces the use of profiles as a reasonable approach to designing new software-hardware co-designs.

We furthermore demonstrate in Section **??** that the profiles generated from SPEC2017 train inputs do generalise effectively to reference inputs, suggesting our proposed technique is able to generalise to unseen inputs. It is also important to note that in the case of load instructions which are unseen in the train inputs, these are never labelled and hence execute as normal. This allows PG-MDP to be more tolerant to workloads with high code coverage variation between inputs.

Lastly we highlight the benefits that profiles are able to provide to this technique over static analysis. Of the four types of load behaviour referred to in Section 3.1, only two of them (types (1) and (3)) could theoretically be captured by perfect alias analysis. Type (2) would further require perfect memory dependence analysis, and there exists no analysis in conventional compiler design that attempts to capture the behaviour in type (4). It is also important to note that the alias and dependence analysis offered by industrial compilers today is far from perfect [**?**]. Whereas stronger analyses do exist [1, 10], these are either only effective in domain-specific languages or do not scale to large programs, making them either less applicable or less feasible. In contrast, profiles allow much greater flexibility and coverage of load behaviour, which pairs well with speculative execution where mistakes will not invalidate program semantics.

4 Evaluation

This section presents the experimental results of applying PG-MDP to SPEC2017 CPU intspeed workloads. We first analyze further detail how PG-MDP shrinks the performance gap between small and large Store Set-like predictors and demonstrate the extent to which the predictor working set can be reduced. This is followed by a breakdown of per-workload IPC gains in the default small core configuration, a power saving for the PHAST predictor on the large core, and lastly an estimation of the impact of limited MDP read ports on dispatch bandwidth for high-load density workloads.

4.1 Experimental Design

We use Gem5 [3] to simulate varying core size configurations. The full configuration for each model is detailed in Table 1. We evaluate the each workload using up to 10 simpoints, with an interval of 100M instructions and a warm-up of 10M. Gem5 is modified to bypass MDP lookups for labelled loads and to avoid creating new entries in the case of a memory order violation. Violating labelled loads still roll back as normal and do not alter program semantics. The Gem5 results are subsequently processed by an extended McPAT [**? ?**], which models XS Store Sets or PHAST as applicable.

The small core configuration represents our intended use case of an area-constrained out-of-order core. The medium core configuration represents a middle ground where a larger MDP can be afforded, but the core is still too small for a more sophisticated predictor like PHAST to be viable. The large core implements PHAST instead of XS Store Sets, to analyze how PG-MDP might impact the state of the art.

Table 1: Parameters of processor components for each simulated model

		Small	Med.	Large
Instruction Window	ROB:	128	256	512
	IQ:	77	154	308
	LQ:	38	77	154
	SQ:	22	54	108
Pipeline Width	Fetch/	4	6	8
	Commit:	8	8	12
	Issue:			
L1 Cache	L1D (KB):	32	64	64
	L1I (KB):	32	32	64
	MSHRs:	8	16	32
L2 Cache	L2 (MB):	2	4	8
	MSHRs:	16	32	64
XS Store Sets	SSIT:	64	256	N/A
	LFST:	32	128	
	Slots:	2	2	
	Clear:	125k	125k	
PHAST	Rows:	N/A	N/A	128
	Assoc:			4
	Tag Bits:			16
TAGE-SC-L Size:		64KB	128KB	
Prefetcher:		DCPT [**?**]		
Store Distance Threshold:		8	13	51

4.2 Predictor Size Performance Gap

Figure 2 shows an extended plot of the SPEC2017 CPU intspeed results in Figure 1, including the medium core configuration as well as the small core. The x-axis represents both the number of SSIT and LFST entries (after being multiplied by an appropriate number of slots). Both cases confirm the initial hypothesis that for a Store Sets-like predictor, a small number of entries can capture most of the performance of a large number of entries for an appropriate predictor working set. Figure 3 shows the reduction in dynamic MDP queries, with an arithmetic average reduction of 77%, demonstrating that only roughly 25% of the original predictor working set needs to be tracked with hardware structures.

In the small core size configuration, 64 table entries with PG-MDP achieves an IPC only 0.46% below that of 1024 entries, with 64 entries on the medium core at 0.72% below 1024 entries. In an area-constrained use case as in the small core, this buys most of the benefit of a larger predictor without any area cost. The medium core represents a case where more area can be spared and so contains a predictor that already delivers most of the potential benefit. Here,

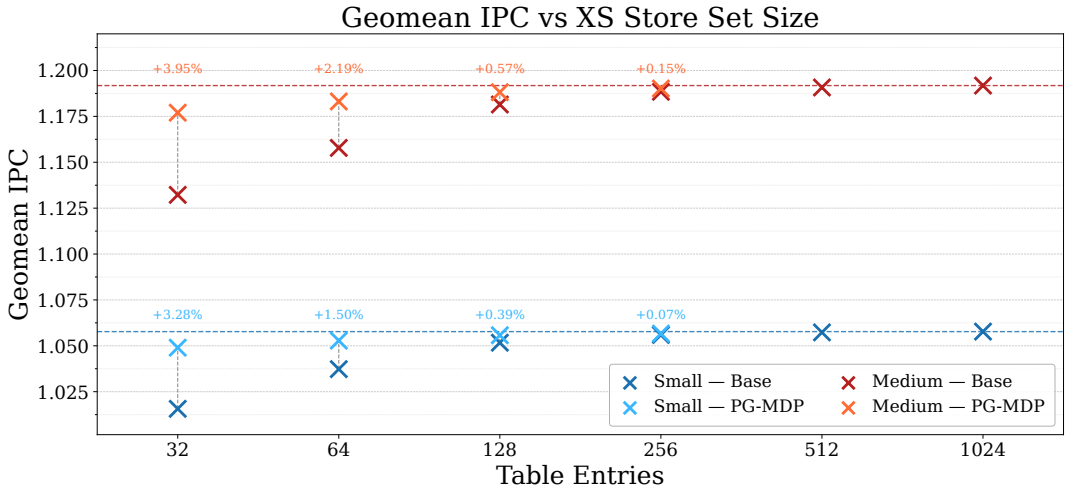

Figure 2: SPEC2017 CPU intspeed geomean IPC with and without PG-MDP for varying XS Store Set sizes. PG-MDP is able to effectively improve small predictor performance to within levels near-equal to large predictors for both a small and medium sized core.

PG-MDP can be used to reduce the area cost while maintaining performance.

4.3 Where Performance Improves

Figure 4 shows the percent IPC changes in each intspeed workload for the small core configuration. It can be seen that IPC gains have an uneven spread across workloads, with some seeing large benefits and others next to none. This loosely correlates to the magnitude of false dependencies before and after seen in Figure 5. The four largest gainers - 625.*x*264_*s* inputs 1 and 2, 641.*leela_s* and 620.*omnetpp_s* - all exhibit relatively higher false dependencies per kilo-instruction in the base case, and see this cut down significantly by over 80% with PG-MDP.

In select cases PND labels can lead to a small performance regression. This is due to loads behaving memory independent on train inputs but exhibiting dependencies on ref inputs and causing an increase in memory order violations, as seen in Figure 6. Although memory order violations are increased in all workloads, they're mostly offset by the reduction in false dependencies. It is worth noting that violations are already rare events, measured in mega-instructions rather than kilo-instructions, so even a 100% increase can often still remain in negligible ranges compared to the number of false dependencies.

The degree to which violations increase is controlled by the threshold value discussed in Section **??**. The optimal threshold may still provide a net performance gain while still causing small regressions in specific workloads, and a larger threshold could instead be chosen to ensure regressions never occur. They could also be mitigated by specialising the PGO technique to each workload rather than the processor, which is discussed further in Section **??**.

4.4 Power Savings & PHAST

Figure **??** shows the total core energy reduction of each workload on the small configuration with PG-MDP applied. Because PG-MDP is a zero-cost optimisation, it can be seen that IPC gains directly translate into correlated power savings with no offset introduced by additional area costs.

Reducing MDP queries should in theory also offer a further power saving. However, we find that XS Store Sets already uses negligible power compared to the LSU and total core, that any reductions don't show up in final figures. Furthermore, XS Store Sets power usage is dominated by the sub-threshold leakage rather than runtime-dynamic cost (i.e. accessing the tables), and so an average 78% reduction in MDP queries only translates to an average 13% reduction in total MDP power.

However, the PHAST predictor incurs a greater power usage due to containing more tables and searching them all in parallel. This makes PHAST's runtime-dynamic power usage about equal to sub-threshold leakage, and takes up a larger proportion of total LSU power usage. This is especially true for larger cores that contain larger predictors. Evaluating PG-MDP on the large core configuration is shown to maintain performance while reducing average PHAST power usage by 40%.

- don't think we need to plot any graphs for the large core as they're not interesting, can just include the numbers - talk about total portion of LSU power, how its still quite small, but maybe future-proofs PHAST better

4.5 Read Port Impact Estimation

Real processors have a limited number of read ports for each predictor component. If the memory dependence predictor has two ports, the processor can dispatch (that is, allocate instructions into out-of-order structures) up to two memory operations per cycle. Gem5 does not model this behaviour by default, as this is a low-level detail that is difficult to faithfully represent at the level of abstraction Gem5 simulates. Given the importance of this aspect to our proposed technique, we provide an estimation of the potential impact on IPC when the number of MDP read ports is limited. The

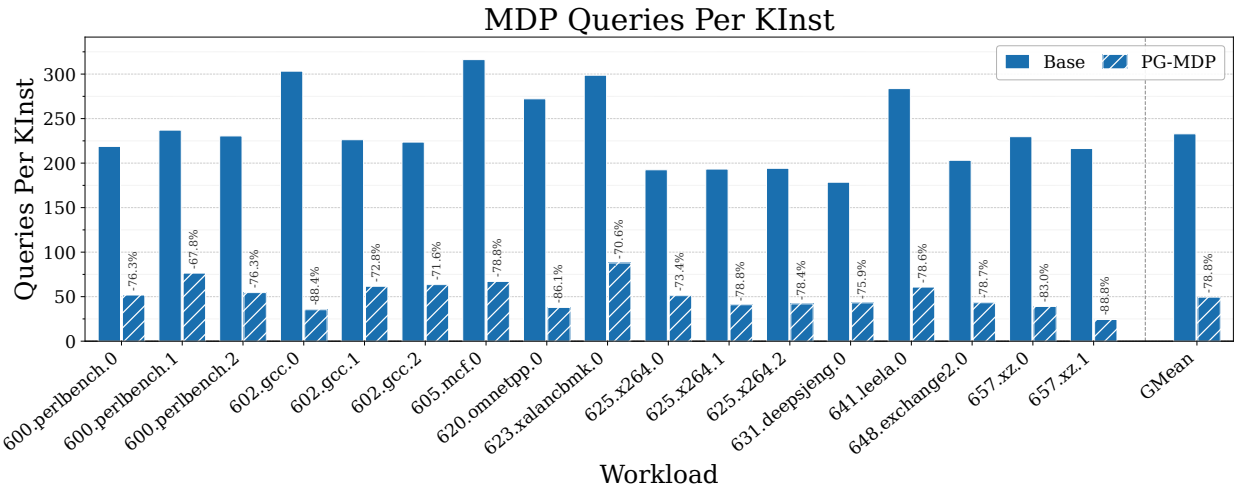

Figure 3: Percent change of memory order violations. Values roughly equal between core sizes.

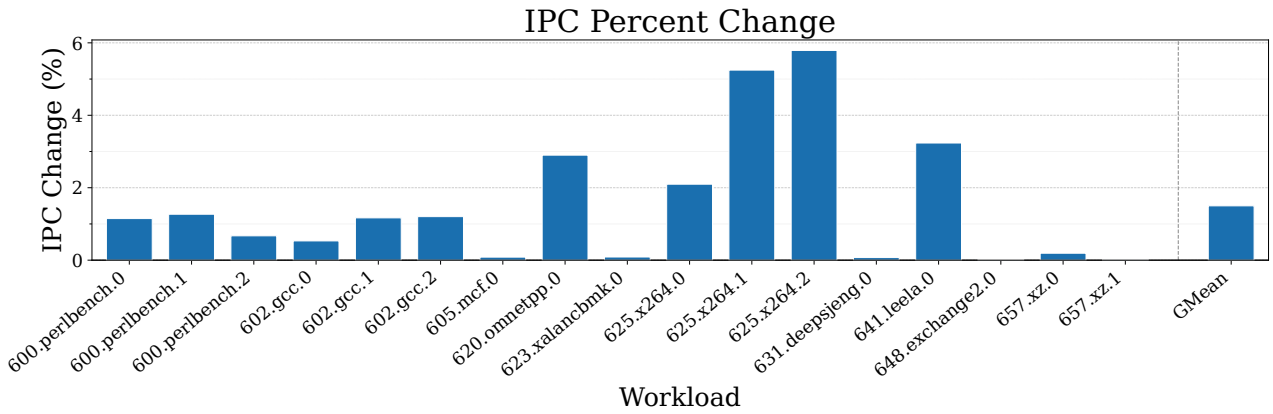

Figure 4: TODO: overview the important trend in each figure caption. IPC % changes for each CPU model on each intspeed workload.

dispatch stage is modelled to block when the number of unlabelled memory operations dispatched within a cycle exceeds the maximum number of read ports, which we model in the small core as 1. It is assumed that predictions always arrive within a single cycle, and so read port usage is reset to zero every cycle.

- insert results here

5 Related Work

Improving Memory Dependence Prediction with Static Analysis [**?**] tackles the same problem as this paper, but using static analysis to determine which load instructions to label as 'PND'. It demonstrates small IPC gains in select cases, but with an overall best-case geomean improvement of less than 0.1%. Furthermore, due to relying on static analysis it also sees performance regressions elsewhere, hurting viability. In comparison, by leveraging profiles we are able to achieve much higher IPC gains while also avoiding regressions. Our technique is also resistant to regressions on larger or more sophisticated MDP algorithms such as PHAST, which [**?**] is not.

Feedback-directed Memory Disambiguation Through Store Distance Analysis [**?**] proposes a similar profile-guided co-design to this paper, but with a more aggressive use case. In their work, the MDP is replaced altogether and load instructions are extended to include 4-bit distance fields indicating the store queue position they're likely to be dependent on. This achieves high accuracy compared to Store Sets, but comes at the cost of higher instruction bandwidth to encode predicted store distances, which is usually infeasible in modern processor designs where instruction bandwidth is critical. Larger processors would also likely require more bits to encode longer store queue distances. Furthermore, [**?**] could be more sensitive to profile accuracy than our work, as loads which do not appear in the profile must be assumed to be memory independent. In contrast, our technique is able to leave unseen loads to execute as normal.

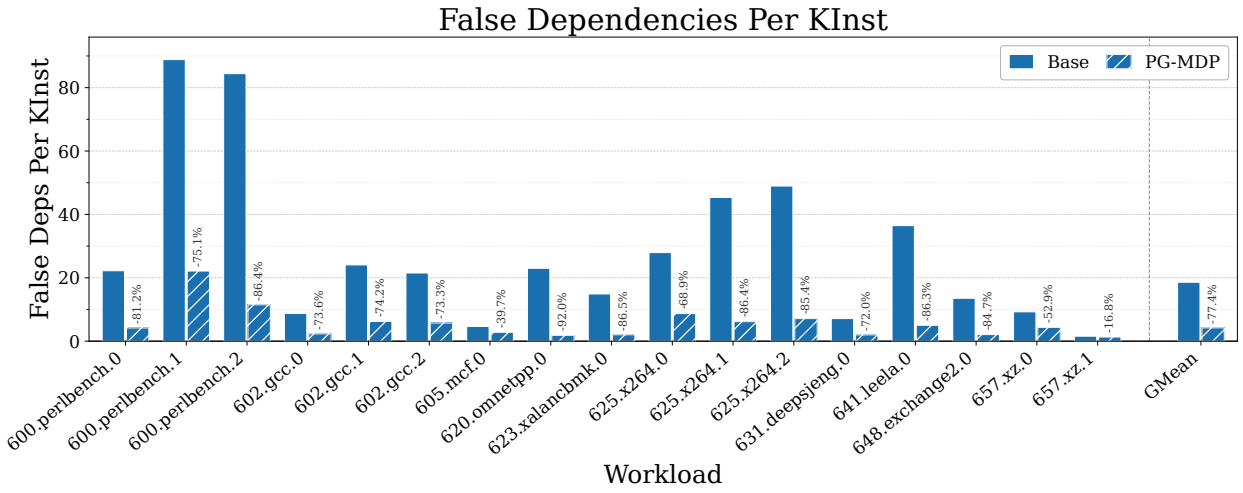

Figure 5: Percent change of false dependencies.

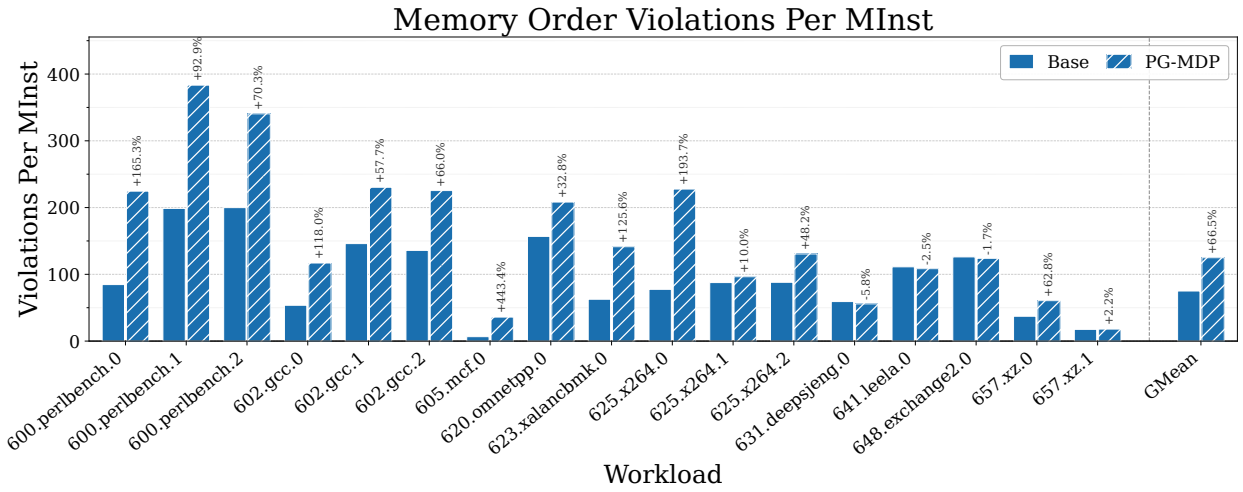

Figure 6: Percent change of memory order violations. Values roughly equal between core sizes.

Software-hardware Cooperative Memory Disambiguation [4] is a binary analysis co-design to reduce load queue pressure. Certain kinds of provably memory independent loads are labelled and prevented from being inserted into the load queue when issued. This technique is able to label 40% of static loads across SPEC2006 floatspeed (but dynamic percentage is unclear). This also incurs additional instruction bandwidth by inserting barrier-like instructions when entering loops to ensure labelled loads are only removed from the load queue when their parent loop reaches a steady state in the pipeline. LSQ entries must also store an additional bit to track whether these custom instructions are in-flight or not. This work presents an interesting proposal for reducing load queue pressure in area-constrained processors that lack space for hardware-based load elimination techniques, but has reduced viability due to currently lacking a mechanism for ensuring memory coherence in multi-core systems.

Constable: Improving Performance and Power Efficiency by Safely Eliminating Load Instruction Execution [**?**] is a pure hardware technique to track and eliminate stable loads which consistently read the same value from memory. This technique is able to eliminate both the memory read and address calculation altogether, greatly improving pipeline efficiency. It also demonstrates a wide coverage a variety of workloads, improving geomean IPC by 5.1%. For larger performance-orientated processors, this technique would most likely be more effective at improving the efficiency of load execution than our work, with a large overlap (albeit not total) of the types of load instructions targeted. However, at an estimated hardware overhead of 12.4KB, this technique may be infeasible for the area-constrained cores we target.

Other Memory Dependence Prediction Algorithms Besides Store Sets and PHAST [2, 5], other MDP algorithms include Store Vectors, MDP-TAGE, and MASCOT [6–9]. Each of these algorithms offer

different trade-offs and advantages. We choose to evaluate XS Store Sets and PHAST because we understand them to be the optimal choices for small and large cores respectively, and that XS Store Sets provides, to the best of our knowledge, the closest representation of MDP algorithms found in real processors.

6 Threats to Validity

This section touches on possible threats to the validity of results presented in this paper.

6.1 Encoding Space for Load Labels

We have proposed new opcodes to differentiate our labelled PND loads from regular loads in the ISA at zero overhead. While possible in theory, this may be difficult to implement in pre-existing and widely used ISAs such as X86, AArch64 and RISCV due to limited available encoding space. Alternate proposals to this problem have been offered in [4]. Further options include introducing some level of overhead to apply our proposed technique, such as extending load opcodes to include a label bit, or reserving a bit in the register operand and adjusting register allocation accordingly.

6.2 Simulated Processor Models

The processor model we simulate specified in Section **??** do not reflect any real world processor configuration. Though we would like to use publicly known parameters from real processors (for instance from sources such as [**?**]), using Gem5 to simulate workloads this can be dangerous, as the features which inform the optimal parameter values in those processors are likely absent in the Gem5 core. As such we design the simulated models used in this paper by the intuition of what is reasonable for Gem5 specifically, while still within realistic area bounds for a real processor in our use case.

6.3 Target Specific Compilation

The technique proposed in this paper assumed target specific compilation is possible in order to select the processor-specific optimal store distance threshold. In many cases though this is not the case. Enterprise software products are often compiled once and shipped to many different platforms. This would make short thresholds invalid, as they would introduce performance regressions in larger processors. A conservatively large threshold could be used for generic compilation, but this would likely also eliminate any performance gains in small processors too. One solution could be for processors known not to benefit from PND labels to ignore the ISA hint and execute labelled loads as normal.

7 Future Work

Per-Workload Threshold Selection could achieve higher performance than per-core thresholds. As the threshold is untied to any architectural state, it is possible to select a different optimal store distance threshold for each individual workload. We chose not to do this for this work to prove that meaningful performance gains were possible with a generic threshold, and that the PG-MDP technique did not require a threshold search for each compiled program. However, there are use cases where this is feasible, and it would be worthwhile to evaluate the potential performance available for the additional compilation cost.

Just-in-Time Compilers (JITs) are a widely used run-time based compilation technique for dynamic languages. PG-MDP could potentially pair well with these compilers for various reasons. Firstly, the additional workflow complexity of profiling can be automated by the run-time optimiser. Secondly, JITs make speculative optimisations about observed program behaviour, and so could aggressively apply PND labels to any loads which read from addresses not recently stored to. Lastly, dynamic languages tend to be implemented with many memory independent pointer-chasing loads. This suggests these workloads could have a high potential for performance gains from PG-MDP.

Serverless Workloads are characterised as a collection of many short-lived workloads which only execute a few times, then not again for a long time with lots of unrelated code in-between. This effectively makes their execution always cold on the target processor, so high performance is difficult to achieve. When memory dependence predictors cannot learn dependencies fast enough to deliver performance gains (which is especially a concern for modern TAGE-like predictors such as PHAST), it may be preferable to disable memory dependence prediction entirely and conservatively stall loads on any unresolved stores. In this case, PG-MDP could potentially deliver substantial IPC gains, even on large cores, by allowing likely-independent loads to still issue immediately.

8 Conclusion

This paper proposed profile-guided memory dependence prediction (PG-MDP), a software-hardware co-design technique to label load instructions via their opcode as 'predict no dependency', and prevent them from making MDP queries to eliminate false dependencies. We showed PG-MDP effectively reduces false memory dependencies at zero hardware cost, and delivers a geomean average of 1.47% IPC gain in a simulated area-constrained core, along with a correlated power saving. We further showed that this technique does not introduce regressions as processor size scales, and can potentially offset the increased energy usage of modern MDP algorithms like PHAST in these processors.

References

[1] [n. d.]. MLIR Affine Dialect. https://mlir.llvm.org/docs/Dialects/Affine/.

[2] George Z. Chrysos and Joel S. Emer. 1998. Memory Dependence Prediction Using Store Sets. In *Proceedings of the 25th Annual International Symposium on Computer Architecture, ISCA*. IEEE Computer Society, 142–153. https://doi.org/10.1109/ISCA.1998.694770

[3] Jason Lowe-Power et al. 2020. The gem5 Simulator: Version 20.0+. https://arxiv.org/abs/2007.03152. arXiv:2007.03152 [cs.AR]

[4] R. Huang, A. Garg, and M. Huang. 2006. Software-hardware cooperative memory disambiguation. In *Proc. HPCA, 2006.* 244–253. https://doi.org/10.1109/HPCA.2006.1598133

[5] Sebastian S. Kim and Alberto Ros. 2024. Effective Context-Sensitive Memory Dependence Prediction. In *30th Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Edinburgh, Scotland.

[6] Karl H. Mose, Sebastian S. Kim, Alberto Ros, Timothy M. Jones, and Robert D. Mullins. 2025. Mascot: Predicting Memory Dependencies and Opportunities for Speculative Memory Bypassing. In *31st Symposium on High Performance Computer Architecture (HPCA)*. IEEE Computer Society, Las Vegas, NV, USA, 59–71.

[7] Arthur Perais and André Seznec. 2017. Storage-Free Memory Dependency Prediction. *IEEE Comput. Archit. Lett.* 16, 2 (Nov. 2017), 149–152. https://doi.org/10.1109/LCA.2016.2628379

[8] Arthur Perais and André Seznec. 2018. Cost effective speculation with the omnipredictor. In *Proceedings of the 27th International Conference on Parallel Architectures and Compilation Techniques, PACT*. ACM, 25:1–25:13. https://doi.org/10.1145/3243176.3243208

[9] Samantika Subramaniam and Gabriel H. Loh. 2006. Store vectors for scalable memory dependence prediction and scheduling. In *12th International Symposium on High-Performance Computer Architecture, HPCA*. IEEE Computer Society, 65–76. https://doi.org/10.1109/HPCA.2006.1598113

[10] Yulei Sui and Jingling Xue. 2016. SVF: interprocedural static value-flow analysis in LLVM. In *Proceedings of the 25th International Conference on Compiler Construction*. ACM, 265–266.

A Examples

Listing 1: 1. Memory Independent Loads: 525.x264_r (Video Compression)

```
// Simplified from x264 CABAC/Quantization tables
const int cabac_context_state[1024] = { /* pre-
    computed static data */ };

void encode_macroblock(int* pixels, int ctx_idx) {

    // Memory Independent Load:
    // The MDP will never see an in-flight store
    // to cabac_context_state.
    // Store distance is infinite.

    int state = cabac_context_state[ctx_idx];

    pixels[0] = apply_transform(pixels[0], state);
}
```

Listing 2: 2. Rarely Dependent Loads: 500.perlbench_r (Perl Interpreter)

```
// Simplified from perlbench hash table operations
struct HashBucket { char* key; int value; };

void update_symbol_table(struct HashBucket* table,
     int hash1, int hash2) {
    // Store to one variable

    table[hash1].value = 100;

    // Rarely Dependent Load:
    // 95%+ of the time, hash1 != hash2, so there
    // is no dependency. However, if the script
    // accesses the same variable or aliases to
    // the same bucket, the load suddenly has a
    // very short store distance.

    int check_val = table[hash2].value;
    if (check_val == 0) allocate_new_symbol();
}
```

Listing 3: 3. Structurally Independent Loads: 556.omnetpp_r (Discrete Event Simulation)

```
  // Simplified from omnetpp message passing
void simulation_loop(struct EventQueue* q) {
    // 1. Store: A module schedules a future event
    // This store retires to the L1 cache quickly

    schedule_event(q, NEW_PACKET_EVENT);

    // ... Millions of cycles of other simulated
    // network activity ...

    process_other_network_nodes();

    // 2. Structurally Independent Load:
    // The load occurs here, but the corresponding
    // store retired millions of instructions ago
    // The store distance is effectively infinite
    // from the perspective of the SQ

    struct Event* next_event = pop_event(q);
}
```

Listing 4: 4. Fast-to-Resolve: 502.gcc_r (C Compiler)

```
// Simplified gcc AST parsing (Stack Access)
void parse_expression() {
    int local_flags = 0; // Fast Store: Address is
                         // Stack Pointer + 4

    // ... a few instructions ...

    // Fast Load: Address is known instantly at
    // dispatch
    // The LSQ can forward the data immediately

    if (local_flags & OPTIMIZE_FLAG) {
        do_optimization();
    }
}
```

Listing 5: 4. Slow-to-Resolve: 505.mcf_r (Combinatorial Optimization)

```
// Simplified mcf network simplex
// (Pointer Chasing)

void update_network(struct Node* n) {
    // Slow Store: The address requires chasing
    // pointers first

    n->child->sibling->potential = 50;

    // Slow Load: Store distance is 1, but the LSQ
    // cannot confirm the aliasing until the long-
    // latency address generation for the store
    // (and the load) is completely resolved.
    // Forwarding is stalled

    int pot = n->child->sibling->potential;
}
```